Fault-Tolerant Distributed Systems – Lab record

Lab 2 – Group G: member 1, member 2, member 3, member 4

1. Claim

Under assumptions $N = 3$ servers, $Q_r = Q_w = 2$, lock quorum $Q_l = 2$; crash-stop failures (no recovery); asynchronous network that may delay, reorder and drop messages (partitions) but not duplicate or corrupt them; each client bound to one server, if at most $f = 1$ server fails by crashing or being cut off from the other two, the service (q4b.py) guarantees: (i) a read that starts after a write/cas completed returns it or a newer value, replicas converge, and no two cas succeed from the same value; (ii) answers to reads for clients of the live majority, and to writes/cas (ok or a definite error 11) unless the key's lock is held by the faulty server. It does not guarantee linearizability: a read overlapping an unfinished write may return the new value and a later read the old one (regular, not atomic, reads).
Evidence: 19/19 linearizable runs of q4b.py (latency 50/rate 50, with n1 crashing, with partitions, with and without latency); each mechanism's necessity shown by a failing run of the step before it (§2); the remaining anomaly reproduced (§3).
Not covered: 2 failures (no quorum); majorities-ring partitions (no working round trip anywhere); recovery without memory; a crashed/cut-off lock holder (key blocked forever, observed in runs/p4-crash-holder, runs/p4-blocked-key); fairness of the lock; valid runs are not proofs (20 s, one hot key).

2. Design and predictions

Evolution (one program per step, code/; every coordinator sends to all servers and waits for the first Q answers):

Part	design	what broke (run)	what we added and why
1 q1	values only; reader returns its own copy first	0/9 linearizable under load: concurrent writes applied in opposite orders, copies diverge (runs/p1-rowa-latency, runs/p1-majority-latency)	versions: servers and readers need an order on values
2 q2	[version, value]; writer asks a read quorum, takes max+1	0/5 + 2/5: concurrent writes get the same version; each server keeps its own (runs/p2-latency: 3 values, version 47)	tie-break on (number, client id): same total order everywhere, no extra message
3 q3	tags [n, client]	10/10 under load; but a read can return a write still at one server, and a later read the old value (runs/p3-inversion, runs/p3-rowa-partition)	kept (no write-back in Parts 1–4); fixed in "going further"
4 q4a	cas = quorum read, compare, quorum write	0/3: two cas(2→1) both read [57,c11] and both succeed (runs/p4-lost-update)	majority lock on the key for cas and write ($2Q_l > N$)
4 q4b	lock: refuse if held; requester releases, random backoff, error 11 after 0.8 s	19/19 linearizable; ≈3.5 locked ops/s per key; a crashed (3 of 7 crash runs) or cut-off holder blocks the key (runs/p4-crash-holder)	(claim above)
+ qx	q3 + write-back on reads	slow link: 0/3 → 3/3; cost: 2nd round trip, ROWA reads need all 5 (runs/px-rowa-crash)	optional step

Part 1 (q1.py, 3 runs per cell; lin = linearizable runs; ok/fail/info of the first run):

Config, scenario	Predicted	Observed	ok/fail/info	Why it differs
ROWA (1,3) no faults / maj. (2,2) / non-int. (1,2)	lin (tiny windows); all	lin 3/3, 3/3, 3/3	125/81/0	–
all three, --latency 50 --rate 50	NOT lin	0/3, 0/3, 0/3	703/282/0	(1,2): divergence, not stale reads
ROWA, n1 crashes at 5 s	lin; no write after 5 s	3/3; 0 writes ok	111/61/32	–
majority / non-int., n1 crashes	lin; all but n1's clients	3/3, 3/3	118/72/7	–
ROWA, partitions	NOT lin; writes blocked	1/3	138/77/3	valid run: 2.3 s partition
majority, partitions	NOT lin (stale own copy)	3/3	13/139/34	window not hit; rings block all
non-int., partitions	NOT lin	0/3	126/80/8	–

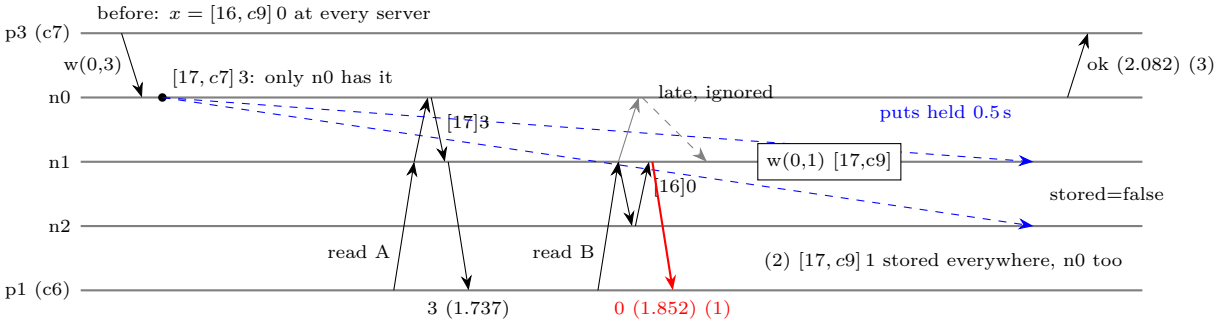
Part 3, Q3 (q3.py, $N = 5$; R/W = reads/writes still completed after the crash at 5 s by clients of live servers):

Config	no faults	n1 crashes	n1, n2 crash	partitions	msgs/op	Part 0 prediction (R / W)
majority (3,3)	lin	lin; R W	lin; R W	lin	6.56	2 / 2 crashes ✓
ROWA (1,5)	lin	lin; R, no W	lin; R, no W	NOT lin	6.22	4 / 0 ✓
(2,4)	lin	lin; R W	lin; R, no W	lin	6.67	3 / 1 ✓
(4,2)	lin	lin; R W	lin; nothing	lin	6.62	1 / 3 ×: writes 1

Our write first reads tags from Q_r servers, so writes tolerate $\min(N - Q_r, N - Q_w)$ crashes: (4,2) completes nothing with 2 crashes. msgs/op is the same for every configuration because every request goes to all $N - 1$ others and all answer; Q only sets how many answers we wait for. Read-heavy service: (2,4): reads wait for the 2 fastest of 5 and survive 3 crashes, writes survive 1; ROWA's cheaper reads cost all write availability at the first crash or partition, and show the inversion. Write-heavy: majority (3,3): writes survive 2 crashes, the maximum; (4,2) does not help with a two-phase write.

3. Evidence

runs/p3-inversion (= store/lin-kv/20261030T152554.648Z; q3.py, SLOW_PUT=n0=0.5: n0's puts are held 0.5 s; not linearizable, key 0; reads of q4b.py are identical).



(1) Read B (p1, after its own read A returned 3) asks n1 and n2, neither of which has the held put, and returns 0: new, then old. The error is already in A: it returned a value stored at 1 server ($< Q_w$), so a later read quorum $\{n1, n2\}$ could miss it. (2) A concurrent write also takes number 17 (its phase 1 asked n1, n2) and wins the tie-break (c9 > c7), even at n0. (3) Write 3 completes although its value is already gone: legal, it overlaps write 1. Classification (key 0, this execution): not linearizable: A precedes B in real time, A needs $w(0, 3)$ before it, B a write of 0 after it, and the only one ($w(0, 0)$, ended 1.492) precedes $w(0, 3)$ (began 1.535). Not sequentially consistent either: p1 read 3 then 0, while p0 read 0 then 3 (program orders), so $w(0, 0)$ must be both before and after $w(0, 3)$. With A and B by different clients and no opposite observation, it would be sequentially consistent (order $w(0, 0)$, B, $w(0, 3)$, A) but still not linearizable.

4. Limitations and open questions

- Reads are regular, not atomic (§3); write-back fixes it (qx.py: slow link 0/3→3/3) at the cost of a second round trip on disagreeing reads, and makes ROWA reads need all N servers.
- A lock held by a crashed or cut-off coordinator is never released: n1 crashed holding {n1, n0} (runs/p4-crash-holder) and no lock on key 0 was granted for the remaining 14 s; a lock request still in flight from the dead n1 was granted to it in 2 more runs; a partition did the same (runs/p4-blocked-key: n0, n1 got 17 error 11, 0 writes/cas). Next: fencing numbers / ballots so a new holder can take over safely; leases need clock assumptions. The lock is also unfair (starvation).
- Not tested: recovery; $N = 5$ with the lock; longer runs and many keys; a targeted test of q2local's stale version after a heal (window never hit). The WSL wall clock jumped back up to 0.7 s in some Part 1 runs: timelines checked against node-log order (verdicts unaffected).
- The kit's inversion-run was not available; we reproduced it with a test hook (SLOW_PUT) and found it naturally in ROWA partition runs.

5. AI log

- Claude Code (Claude Opus 5.5) was asked to "complete Lab 2" and document everything: an answer directly request, against the guide's think-first workflow (Part 0 and the group design questions); noted here and in docs/ai-log.md.

- It wrote each part's design in the program header before coding and implemented only that design (one file per step); mechanisms it considered early (versions in Part 1, tie-break in Part 2, write-back, leases) were deferred to the part that asks for them.
- Corrected by the runs, not by the assistant: q2local's first valid run was not "fewer ties" (it had 112); no pure stale read exists in any Part 1 run (non-intersection shows only with partitions); crash and partition runs of the lock must be on top of latency.
- (Group: add what you accepted, changed or rejected.)

6. Contributions

- member 1: …
- member 2: …
- member 3: …
- member 4: …